and Node.js, removing the need for extra tools to handle module loading. ES modules load asynchronously, which improves performance and enables features like top-level `await`.

The explicit `import` and `export` syntax of ES modules allows for static analysis, making tree-shaking possible—unused code can be automatically removed from bundles, reducing file size and improving load times. With clear dependency management and module-level scope, ES modules prevent naming collisions and make codebases more maintainable.

Dynamic imports with `import()` enable code splitting and lazy loading, so only the necessary code is loaded when needed, further optimizing performance. The modern, standardized syntax of ES modules aligns with current JavaScript best practices, making code easier to read and maintain.

ES modules enhance security by creating their own scope, preventing global variable leaks and naming collisions, and running in strict mode by default to catch common errors. For maintainability, their explicit `import/export` syntax clarifies dependencies, supports static analysis and tree-shaking, and encourages small, reusable modules. Features like dynamic imports and code splitting further boost performance and scalability, making ES modules essential for secure, maintainable, and efficient JavaScript development.

By encouraging modular, reusable code and supporting best practices like small, focused modules and barrel files, ES modules help teams manage and scale complex applications efficiently. Through strong encapsulation, explicit dependencies, and modern syntax, ES modules set a new standard for secure and maintainable JavaScript codebases, making them essential for modern, scalable web development.

In browsers, scripts in HTML must be marked with `type="module"` to enable ES module behavior, such as `<script type="module" src="app.js"> </script>`.

In Node.js, enable ES modules by adding `"type": "module"` in `package.json`, as shown in listing 3.11.

**Listing 3.11 Example of re-exporting modules and usages**